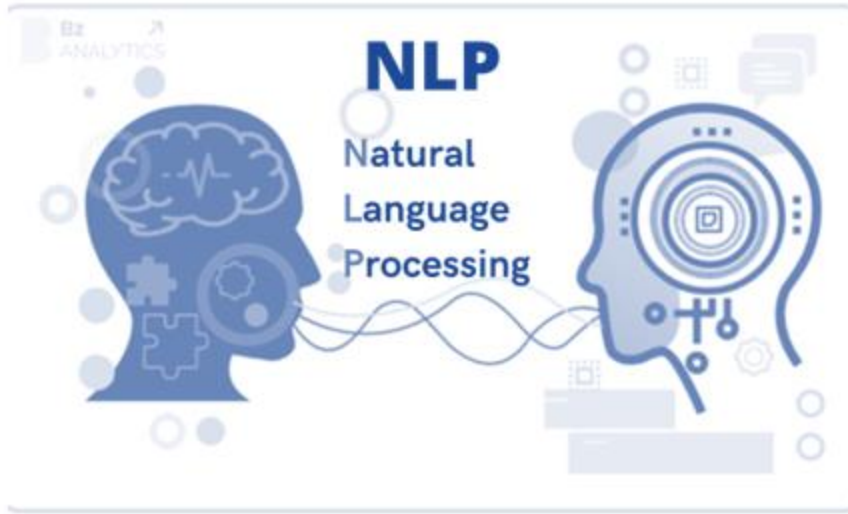


Outline:

1	What is NLP
2	NLP Pipeline
3	Regular Expression
4	Tokenization
5	Case Folding
6	Stemming
7	Lemmatization
8	Stop Word

Outline:

9	Part-of-Speech
10	Named Entity Recognition
11	Spell Correction

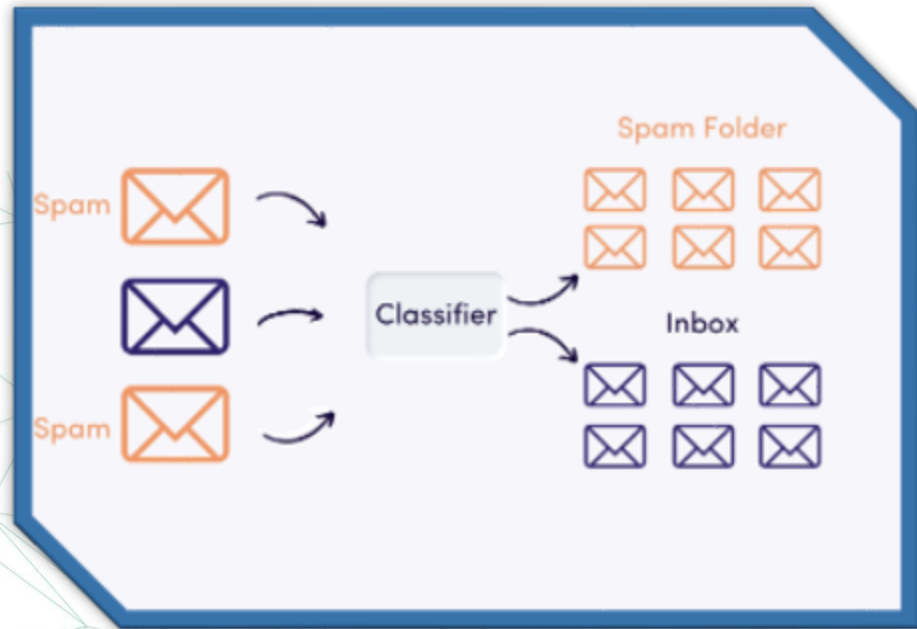


Natural language processing (NLP)

Session one

What is NLP ?

Natural language processing (NLP) is a subfield of computer science and **artificial intelligence (AI)** that uses machine learning to enable computers to understand and communicate with human language.



Applications In NLP :

Text Classification



Sentiment Analysis



Text Similarity



Text Summarization



Machine Translation



Chatbots

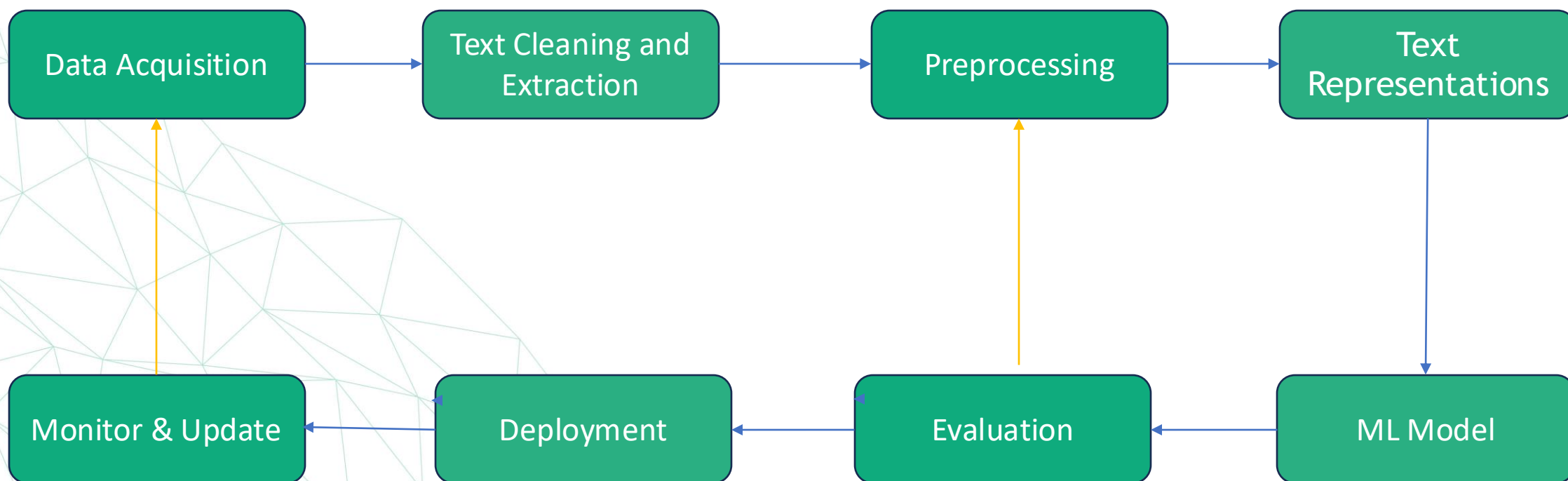


What is NLP ?

- There are some terms we need to agree on:

- **Corpus** → Paragraph
- **Document** → Sentences
- **Vocabulary** → Unique Words

NLP Pipeline

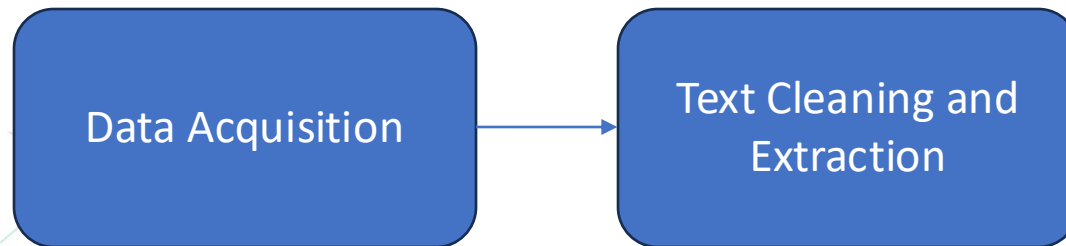


NLP Pipeline

Data Acquisition

- CSV & Excel
- Database
- JSON
- Text Files
- PDF

NLP Pipeline



```
{  
  "title": "camtasia closing and crashing during batch production",  
  "description": "So here's another post complaining about something I've  
    read a couple of times before:\n\n    During video production  
    (though mostly batch production as that's what I usually use),  
    Camtasia loves to cras. The crash is basically..",  
  "creator": "Vladimir Putin",  
  "create_ts": "3/23/2022 10:30 AM EST",  
  "severity": "High"  
}
```

```
{  
  "text": "camtasia closing and crashing during batch production. So here's  
    another post complaining about something I've read a couple of  
    times before:\n\n    During video production (though mostly batch  
    production as that's what I usually use), Camtasia loves to crash.  
    The cras is basically..",  
  "severity": "High"  
}
```

Reg[ular] Ex[pression]*

Text Cleaning using Regex :

- We use it to extract a specific pattern from the text.
- This can be implemented in vision using something called optical character recognition (OCR).
- Of course, it can't be saved, so that's why we're going to this website <https://regex101.com/>

Regular expression :

- Python has a built-in package called re, which can be used to work with Regular Expressions.
- Import the re module:

```
import re
```



RegEx Functions :

Function

findall

search

split

sub

Description

Returns a list containing all matches

Returns a Match object if there is a match anywhere in the string

Returns a list where the string has been split at each match

Replaces one or many matches with a string

The findall() Function:

The `findall()` function returns a list containing all matches.

```
1 import re
2 txt = "The Course in Egypt is one of the best places. (Course)"
3 x = re.findall("Course", txt)
4 print(x)
```

```
['Course', 'Course']
```

If no matches are found, **an empty list is returned**

```
1 import re
2 txt = "The AI for everyone"
3 x = re.findall("Course", txt)
4 print(x)
```

```
[]
```

The sub() Function :

The `sub()` function replaces the matches with the text of your choice

```
1 import re
2 txt = "The Course in Egypt is one of the best places. (Course)"
3 x = re.sub("o", "9", txt)
4 print(x)
```

The C9urse in Egypt is 9ne 9f the best places. (C9urse)

You can control the number of replacements by specifying the `count` parameter

```
1 import re
2 txt = "The Course in Egypt is one of the best places. (Course)"
3 x = re.sub("o", "9", txt, 1)
4 print(x)
5
```

The C9urse in Egypt is one of the best places. (Course)

The search() Function:

The `search()` function searches the string for a match, and returns a Match object if there is a match. If there is more than one match, only the first occurrence of the match will be returned:

```
import re

txt = "The rain in Spain"
x = re.search("\s", txt)

print("The first white-space character is located in position:", x.start())
```

The first white-space character is located in position: 3

If no matches are found, the value `None` is returned:

```
import re

txt = "The rain in Spain"
x = re.search("Portugal", txt)
print(x)
```

None



The split() Function:

The split() function returns a list where the string has been split at each match

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)
```

```
['The', 'rain', 'in', 'Spain']
```

You can control the number of occurrences by specifying the **maxsplit** parameter

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt, 1)
print(x)
```

```
['The', 'rain in Spain']
```


Match ranges in regex :

Regex uses ranges to match sequential groups of characters, like uppercase English letters, without having to write out each letter individually. This makes it easier and less tedious to specify such patterns in your expressions.

RANGE

[A-Z]

[a-z]

[0-9]

MATCHES

uppercase letters

lowercase letters

Any digit

/regex?/

Examples

```
import re
input_string = 'Hello World 123'

uppercase_letters = re.findall(r'[A-Z]', input_string)
print("Uppercase letters:", uppercase_letters)
```

Uppercase letters: ['H', 'W']

```
import re
input_string = 'Hello World 123'

# Lowercase letters
lowercase_letters = re.findall(r'[a-z]', input_string)
print("Lowercase letters:", lowercase_letters)
```

Lowercase letters: ['e', 'l', 'l', 'o', 'o', 'r', 'l', 'd']

Examples

```
import re
input_string = 'Hello World 123'

# Digits
digits = re.findall(r'[0-9]', input_string)
print("Digits:", digits)
```

Digits: ['1', '2', '3']

```
1 import re
2
3 text = "There are 3 apples, 12 bananas, and 7 oranges."
4 matches = re.findall(r'[0-3]', text)
5 print(matches)
6
```

['3', '1', '2']

Special Sequences

RANGE

.

\w

\W

\d

\D

\s

\S

MATCHES

anything except newline

word character

non-word character

digits

non-digits

space, tab, newlines

non whitespace characters

Examples

```
import re
```

```
text = "Hello World_123"
```

```
pattern = re.compile(r'\w')
```

```
matches = pattern.findall(text)
```

```
print("Matches any word character :", matches)
```

Matches any word character : ['H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd', '_', '1', '2', '3']

```
import re
```

```
# Sample text
```

```
text = "Hello World_123!"
```

```
# Match non-word characters
```

```
pattern = re.compile(r'\W')
```

```
matches = pattern.findall(text)
```

```
print("Match non-word characters", matches)
```

Match non-word characters [' ', '!']

Examples

```
import re

# Sample text
text = "Hello 123 World 456!"

# Match non-digits
pattern = re.compile(r'\D')
matches = pattern.findall(text)
print(matches)
```

['H', 'e', 'l', 'l', 'o', ' ', ' ', 'W', 'o', 'r', 'l', 'd', ' ', '!', '']

```
import re

# Sample text
text = "Hello 123 World 456"

# Match digits
pattern = re.compile(r'\d')
matches = pattern.findall(text)
print(matches)
```

['1', '2', '3', '4', '5', '6']

Examples

```
import re

# Sample text
text = "Hello World Python"

# Match non-whitespace characters
pattern = re.compile(r'\S')
matches = pattern.findall(text)
print(matches)
```

```
import re

# Sample text
text = "Hello\tWorld\nPython"

# Match whitespace characters
pattern = re.compile(r'\s')
matches = pattern.findall(text)
print(matches)
```

['H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd', 'P', 'y', 't', 'h', 'o', 'n'] ['\t', '\n']

```
import re
text = "Hello, world\n!"
pattern = r" ,"
matches = re.findall(pattern, text)
print(matches)
```

['H', 'e', 'l', 'l', 'o', ',', ' ', 'w', 'o', 'r', 'l', 'd', '!']

Metacharacters

RANGE

*

?

+

{n}

{n, }

{n, m}

MATCHES

0 or more

0 or 1

1 or more

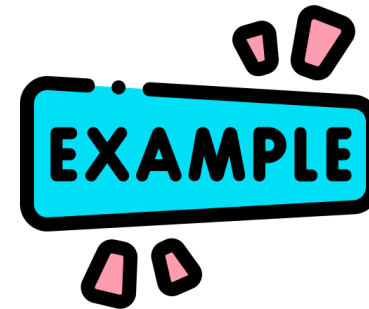
exactly n times

n or more times

n to m times
inclusive

/regex?/

Examples



```
import re

pattern = r"ab*"
test_string = "ababab"
result = re.findall(pattern, test_string)
print(result)  # Output: ['ab', 'ab', 'ab']
```

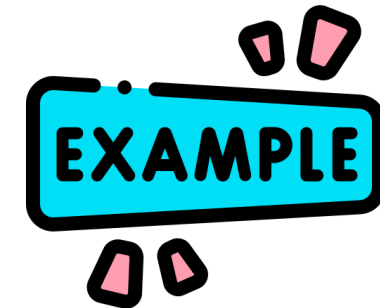
['ab', 'ab', 'ab']

```
import re

pattern = r"ab+"
test_string = "abb"
result = re.findall(pattern, test_string)
print(result)  # Output: ['abb']
# +: Matches one or more occurrences of the preceding element.
```

['abb']

Examples



```
import re

pattern = r"ab?"
test_string = "a"
result = re.findall(pattern, test_string)
print(result) # Output: ['a']
#?: Matches zero or one occurrence of the preceding element.
```

['a']

```
import re

pattern = r"ab{2}"
test_string = "abb"
result = re.findall(pattern, test_string)
print(result) # Output: ['abb']
# {n}: Matches exactly n occurrences of the preceding element.
```

['abb']

Examples

```
import re

pattern = r"ab{2,3}"
test_string = "abbb"
result = re.findall(pattern, test_string)
print(result) # Output: ['abb']
```

{n, m}: Matches between n and m occurrences of the preceding element, inclusive.

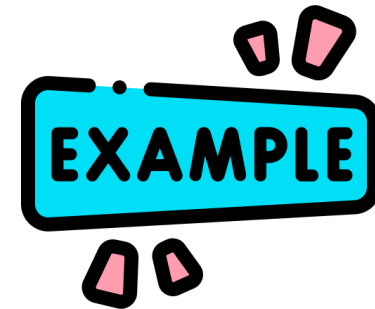
['abbb']

```
import re

pattern = r"ab{2,}"
test_string = "abbbbb"
result = re.findall(pattern, test_string)
print(result) # Output: ['abbbbb']
```

{n,}: Matches n or more occurrences of the preceding element.

['abbbbb']



Try To Solve

Write a regular expression to match a phone number in the format "(XXX) XXX-XXXX"
From “call me at (123) 456-7890”

Try To Solve

Write a regular expression to match a phone number in the format "(XXX) XXX-XXXX"
From "call me at (123) 456-7890"

```
import re

phone_number_pattern = r"\(\d{3}\) \d{3}-\d{4}"
test_string = "Call me at (123) 456-7890"
matches = re.findall(phone_number_pattern, test_string)
print(matches)
```

Try To Solve

How can you extract all email addresses from a given text using regular expressions in Python?

“Contact us at email@example.com or info@example.com”

Try To Solve

How can you extract all email addresses from a given text using regular expressions in Python?

“Contact us at email@example.com or info@example.com”

```
import re

text = 'Contact us at email@example.com or info@example.com'
pattern = r'\b\w+@\w+\.com'
emails = re.findall(pattern, text)
print(emails)
```


Python Regular Expressions

SPECIAL CHARACTERS

^ | Matches the expression to its right at the start of a string. It matches every such instance before each **\n** in the string.

\$ | Matches the expression to its left at the end of a string. It matches every such instance before each **\n** in the string.

. | Matches any character except line terminators like **\n**.

**** | Escapes special characters or denotes character classes.

A|B | Matches expression **A** or **B**. If **A** is matched first, **B** is left untried.

+ | Greedily matches the expression to its left 1 or more times.

***** | Greedily matches the expression to its left 0 or more times.

? | Greedily matches the expression to its left 0 or 1 times. But if **?** is added to qualifiers (**+**, *****, and **?** itself) it will perform matches in a non-greedy manner.

{m} | Matches the expression to its left **m** times, and not less.

{m,n} | Matches the expression to its left **m** to **n** times, and not less.

{m,n}? | Matches the expression to its left **m** times, and ignores **n**. See **?** above.

CHARACTER CLASSES

(A.K.A. SPECIAL SEQUENCES)

\w | Matches alphanumeric characters, which means **a-z**, **A-Z**, and **0-9**. It also matches the underscore, **_**.

\d | Matches digits, which means **0-9**.

\D | Matches any non-digits.

\s | Matches whitespace characters, which include the **\t**, **\n**, **\r**, and space characters.

\S | Matches non-whitespace characters.

\b | Matches the boundary (or empty string) at the start and end of a word, that is, between **\w** and **\W**.

\B | Matches where **\b** does not, that is, the boundary of **\w** characters.

\A | Matches the expression to its right at the absolute start of a string whether in single or multi-line mode.

\Z | Matches the expression to its left at the absolute end of a string whether in single or multi-line mode.

SETS

[] | Contains a set of characters to match.

[amk] | Matches either **a**, **m**, or **k**. It does not match **amk**.

[a-z] | Matches any alphabet from **a** to **z**.

[a\ -z] | Matches **a**, **-**, or **z**. It matches **-** because **** escapes it.

[a-] | Matches **a** or **-**, because **-** is not being used to indicate a series of characters.

[-a] | As above, matches **a** or **-**.

[a-z0-9] | Matches characters from **a** to **z** and also from **0** to **9**.

[(+*)] | Special characters become literal inside a set, so this matches **(**, **+**, *****, and **)**.

^[abs] | Adding **^** excludes any character in the set. Here, it matches characters that are not **a**, **b**, or **s**.

GROUP S

() | Matches the expression inside the parentheses and groups it.

(?) | Inside parentheses like this, **?** acts as an extension notation. Its meaning depends on the character immediately to its right.

(?PAB) | Matches the expression **AB**, and it can be accessed with the group name.

(?aiLmsux) | Here, **a**, **i**, **L**, **m**, **s**, **u**, and **x** are flags:

- a** — Matches ASCII only
- i** — Ignore case
- L** — Locale dependent
- m** — Multi-line
- s** — Matches all
- u** — Matches unicode
- x** — Verbose

(?:A) | Matches the expression as represented by **A**, but unlike **(?PAB)**, it cannot be retrieved afterwards.

(?#...) | A comment. Contents are for us to read, not for matching.

A(?:=B) | Lookahead assertion. This matches the expression **A** only if it is followed by **B**.

A(?:!B) | Negative lookahead assertion. This matches the expression **A** only if it is not followed by **B**.

(?<=B)A | Positive lookbehind assertion. This matches the expression **A** only if **B** is immediately to its left. This can only

matched fixed length expressions.

(?<!B)A | Negative lookbehind assertion.

This matches the expression **A** only if **B** is not immediately to its left. This can only matched fixed length expressions.

(?P=name) | Matches the expression matched by an earlier group named "name".

(...)\1 | The number **1** corresponds to the first group to be matched. If we want to match more instances of the same expression, simply use its number instead of writing out the whole expression again. We can use from **1** up to **99** such groups and their corresponding numbers.

POPULAR PYTHON RE MODULE

FUNCTION S

re.findall(A, B) | Matches all instances of an expression **A** in a string **B** and returns them in a list.

re.search(A, B) | Matches the first instance of an expression **A** in a string **B**, and returns

it as a re match object.

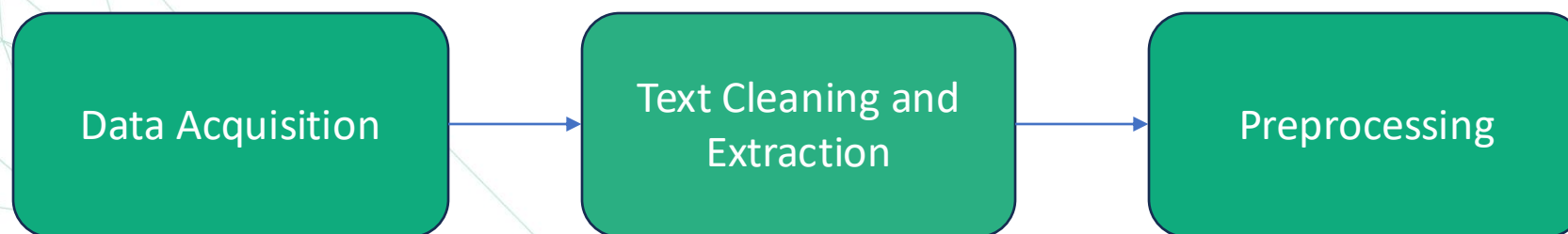
re.split(A, B) | Split a string **B** into a list using the delimiter **A**.

re.sub(A, B, C) | Replace **A** with **B** in the string **C**.

Cheat sheet on Python RegEx:

- <https://www.dataquest.io/wp-content/uploads/2019/03/python-regular-expressions-cheat-sheet.pdf>

NLP Pipeline





"camtasia closing and crashing during batch production. So here's another post complaining about something I've read a couple of times before: During video production (though mostly batch production as that's what I usually use), Camtasia loves to crash. The crash is basically.."

Sentence
Tokenization

"camtasia closing and crashing during
"So here's another post complaining
"The crash is basically.."

What is Tokenization?

Word
Tokenization

Stemming &
Lemmatization

camtasia

close

crash

camtasia

closing

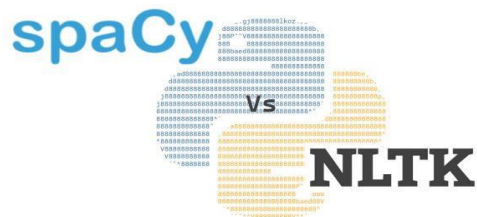
crashing

What is tokenization ?

We usually start an NLP project with a large body of text, called a **corpus**. This could be a collection of tweets, website reviews or transcriptions of films, for example. We need to **pre-process** our corpus to give it enough **structure** to be used in a machine learning model and tokenization is the most common first step.

Tokenization :

- Is the process of breaking down a corpus into **tokens**. The procedure might look like **segmenting** a piece of text into sentences and then further segmenting these sentences into individual **words, numbers and punctuation**, which would be tokens.



“He likes to run.”



["He", "likes", "to", "run", "."]

Input Text

Tokenization is one of the first step in any NLP pipeline. Tokenization is nothing but splitting the raw text into small chunks of words or sentences, called tokens.

Word Tokenization

Tokenization	is	one	of
the	first	step	in
any	NLP	pipeline	Tokenization
is	nothing	but	splitting
the	raw	text	into
small	chunks	of	words
or	sentences	called	tokens

Sentence Tokenization

Tokenization is one of the first step in any NLP pipeline
Tokenization is nothing but splitting the raw text into small chunks of words or sentences, called tokens

Tokenization:

Tokenization is a process to covert either a paragraph or a sentences into tokens if I convert a paragraph into tokens , that basically means I'm converting a paragraph into sentences I can also convert a paragraph into words So if I'm converting into a words , single single words becomes a **token** and if I'm trying to convert a paragraph into sentence every sentence will be a **token** .

Tokenization using NLTK

NLTK is a popular Python library for working with human language data. It provides tools for tasks like **tokenization**, **stemming**.

Making it great for natural language processing (NLP) projects.

- First Download NLTK and punkt to apply **tokenizer**:
- pip install nltk

```
1 import nltk
2 # Download the 'punkt' resource (tokenizer)
3 nltk.download('punkt')
4
```

```
[5] 1 from nltk.tokenize import sent_tokenize
2 corpus="""Hello Welcome,to Mohamed Atef NLP Tutorials.
3 Please do watch the entire course. to become expert in NLP.
4 """
5 documents=sent_tokenize(corpus)
6 documents
```

```
['Hello Welcome,to Mohamed Atef NLP Tutorials.',
 'Please do watch the entire course.',
 'to become expert in NLP.']
```

```
4 from nltk.tokenize import word_tokenize
```

```
1 print(word_tokenize(corpus))
```

```
['Hello', 'Welcome', ',', 'to', 'Mohamed', 'Atef', 'NLP', 'Tutorials', '.', 'Please', 'do', 'watch', 'the', 'entire', 'course', '.', 'to', 'become', 'expert', 'in', 'NLP', '.']
```

Sentence
Tokenization

Word
Tokenization

Tokenization using spaCy

we load a **statistical model** of the English language **trained on web articles**. Its capabilities include tokenization, among other things.

Sentence Tokenization

```
1 import spacy
2
3 # Load the SpaCy model
4 nlp = spacy.load("en_core_web_sm")
5
6 # Define your text
7 text = """Hello Welcome,to Mohamed Atef NLP Tutorials.
8 Please do watch the entire course. to become expert in NLP.
9 """
10 # Process the text
11 doc = nlp(text)
12 # Tokenize into sentences
13 sentences = [sent for sent in doc.sents]
14 print("Sentence Tokens:", sentences)
15
```

```
Sentence Tokens: [Hello Welcome,to Mohamed Atef NLP Tutorials.
, Please do watch the entire course., to become expert in NLP.
]
```

Word Tokenization

```
1 import spacy
2
3 # Load the SpaCy model
4 nlp = spacy.load("en_core_web_sm")
5
6 # Define your text
7 text = """Hello Welcome,to Mohamed Atef NLP Tutorials.
8 Please do watch the entire course. to become expert in NLP.
9 """
10
11 # Process the text
12 doc = nlp(text)
13
14 # Tokenize into words
15 words = [token for token in doc]
16 print("Word Tokens:", words)
17
```

```
Word Tokens: [Hello, Welcome, ,, to, Mohamed, Atef, NLP, Tutorials, .,
, Please, do, watch, the, entire, course, ., to, become, expert, in, NLP, .,
]
```

Case folding using spaCy



1. This is the act of converting every token to be uniformly **lower case** or **upper case**.
2. This can be beneficial because it will **reduce the number of unique tokens** in a corpus, i.e. the size of the **vocabulary**, hence make the processing of these tokens more memory and computational efficient The downside however is **information loss**.
3. or example "Green" (name) has a different meaning to "green" (colour) but both would get the **same token** if case folding is applied. Whether it makes sense to use **case folding depends on the application** (is speed or accuracy more important).

Case folding

```
# Import spacy library
import spacy
```

```
# Load language model
nlp = spacy.load("en_core_web_sm")
```

To case fold to lower cases we can use the `.lower` attribute.

```
# Tokenize
s = "The train to London leaves at 10am on Tuesday."
doc = nlp(s)

# Case fold
print([t.lower_ for t in doc])
```

```
['the', 'train', 'to', 'london', 'leaves', 'at', '10', 'am', 'on', 'tuesda', 'y', '.']
```

We might want to be **more granular** and only case fold if certain conditions are met. For example, we could **skip the first word** in a sentence.

```
# Conditional case folding
print([t.lower_ if not t.is_sent_start else t.text for t in doc])
```

```
['The', 'train', 'to', 'london', 'leaves', 'at', '10', 'am', 'on', 'tuesda', 'y', '.']
```



"camtasia closing and crashing during batch production. So here's another post complaining about something I've read a couple of times before: During video production (though mostly batch production as that's what I usually use), Camtasia loves to crash. The crash is basically.."

Sentence
Tokenization

"camtasia closing and crashing during
"So here's another post complaining
"The crash is basically.."

What are Stemming
and Lemmatization?

Word
Tokenization

Stemming &
Lemmatization

camtasia

close

crash

camtasia

closing

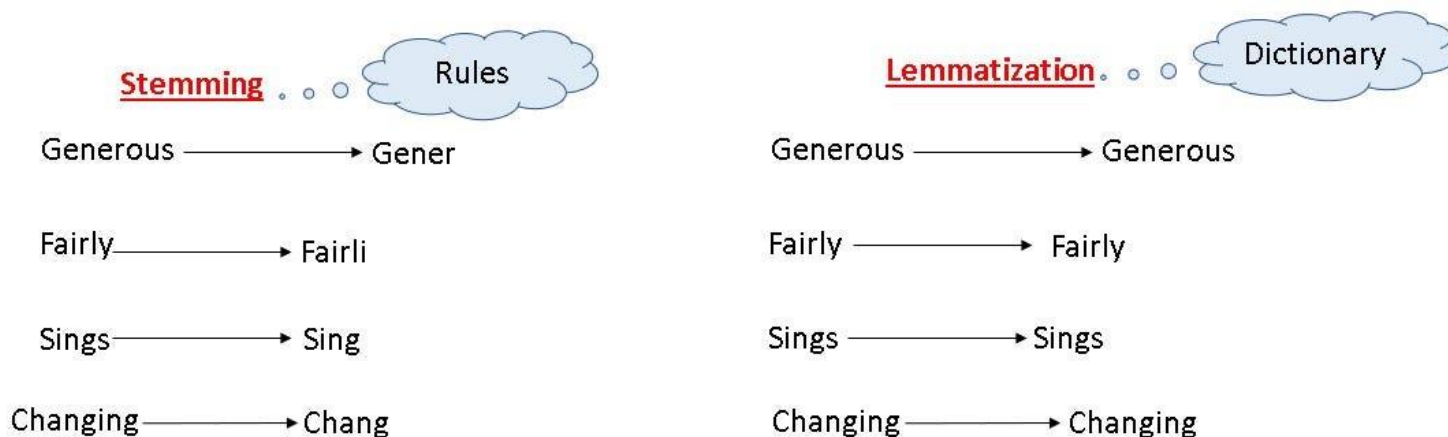
crashing

Stemming

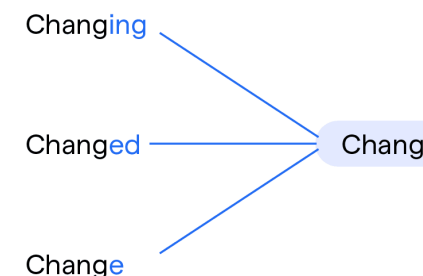
Stemming is the act of **reducing a word to its stem** by **removing suffixes** and sometimes prefixes depending on the language.

For example, the words "developed" and "developing" both have the stem "develop".

While this technique also reduces the size of the vocabulary, it can result in **invalid words**, for example "studies" might be stemmed to "studi". For this reason, stemming is rarely used these days. It turns out there is a **better alternative**, called **lemmatization**, which we'll look at next.



Stemming



Stemming

have 3 types of stemming, each type differs from the next in that it is more **aggressive**, meaning it applies

- rules more to reduce the number of unique words it returns.

Porter Stemming Algorithm

It is less aggressive in stemming and keeps more of the original word than some of the other algorithms.

Porter stemmers only stem English words

Snowball Stemming (Porter2 stemming)

This algorithm is more aggressive and efficient than the Porter algorithm. It applies a larger set of rules to reduce words to their base form.

the Snowball stemmer can stem texts in a number of other Roman script languages, such as Dutch, German, French, and even Russian it is a better version of the Porter Stemmer since some issues of it were fixed in this stemmer.

Lancaster Stemming

This algorithm is the most aggressive among the three. It uses many rules to reduce a word to its base form.

Lancaster Stemming only stem English words

Stemming Code :

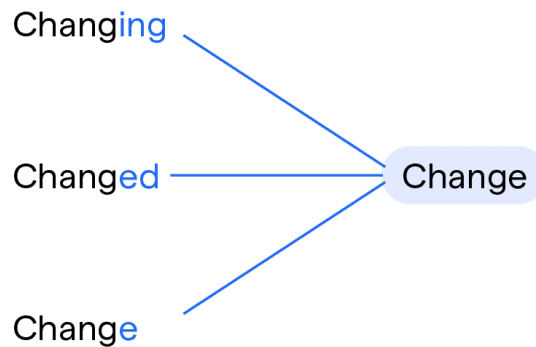
```
1 import nltk
2 from nltk.stem import PorterStemmer, SnowballStemmer, LancasterStemmer
3
4 porter_stemmer = PorterStemmer()
5 snowball_stemmer = SnowballStemmer("english")
6 lancaster_stemmer = LancasterStemmer()
7
8 words = ['cared', 'university', 'fairly', 'easily', 'singing',
9          'sings', 'sung', 'singer', 'sportingly', 'congratulations']
10
11 porter_stems = [porter_stemmer.stem(word) for word in words]
12
13 snowball_stems = [snowball_stemmer.stem(word) for word in words]
14
15 lancaster_stems = [lancaster_stemmer.stem(word) for word in words]
16
17 print("Original words:", words)
18 print("Porter Stemmer:", porter_stems)
19 print("Snowball Stemmer:", snowball_stems)
20 print("Lancaster Stemmer:", lancaster_stems)
21 # congratulate
```

```
Original words: ['cared', 'university', 'fairly', 'easily', 'singing', 'sings', 'sung', 'singer', 'sportingly', 'congratulations']
Porter Stemmer: ['care', 'univers', 'fairli', 'easili', 'sing', 'sing', 'sung', 'singer', 'sportingli', 'congratul']
Snowball Stemmer: ['care', 'univers', 'fair', 'easili', 'sing', 'sing', 'sung', 'singer', 'sport', 'congratul']
Lancaster Stemmer: ['car', 'univers', 'fair', 'easy', 'sing', 'sing', 'sung', 'sing', 'sport', 'congrat']
```

Lemmatization

- Lemmatization reduces a word down to its **lemma**, i.e. dictionary form.
- While this is similar to stemming, it also takes into account things like **tenses** and **synonyms**. For example, the words "did", "done" and "doing" would be converted to the base form "do".

Lemmatization



“He felt better today.”



“He feel well today.”

Lemmatization

- It also takes into account whether a word is a **noun, verb or adjective** on deciding whether to lemmatize. For example, it might not modify some adjectives so not to change their meaning. ("energetic" is different to "energy").
- Lemmatization is generally preferred to stemming because it is **more accurate and robust** while still offering the same benefit of vocabulary size reduction. It does however remove your ability to distinguish different tenses, which may be important for some applications

Lemmatization Using NLTK and SpaCy

```
2 from nltk.stem import WordNetLemmatizer
3 import nltk
4 nltk.download('wordnet')
5
6 lemmatizer = WordNetLemmatizer()
7
8 words = ['cats', 'running', 'better', 'flies', 'congratulations']
9 for word in words:
10     print(f"{word}: {lemmatizer.lemmatize(word)}")
11
```

[nltk_data] Downloading package wordnet to /root/nltk_data...

cats: cat
running: running
better: better
flies: fly
congratulations: congratulation

```
# Tokenize
s = "She was the fastest swimmer."
doc = nlp(s)
```

We can view the lemmatization using the `.lemma_` attribute.

```
# Lemmatization
print([(t.text,t.lemma_) for t in doc])
```

```
[('She', 'she'), ('was', 'be'), ('the', 'the'), ('fastest', 'fast'), ('swi  
mmer', 'swimmer'), ('.', '.')]

```

"camtasia closing and crashing during batch production. So here's another post complaining about something I've read a couple of times before: During video production (though mostly batch production as that's what I usually use), Camtasia loves to crash. The crash is basically.."

Sentence
Tokenization

"camtasia closing and crashing during
"So here's another post complaining
"The crash is basically.."

"The food was delicious!"



["food", "delicious", "!"]

Remove Stop
Words

What is Stop Words?

camtasia

close

crash

Stemming &
Lemmatization

Word
Tokenization

camtasia

closing

crashing

Stop word removal

- Stop words are words that **appear commonly** but **carry little information**. Examples include, "a", "the", "of", "an", "this", "that". Similar to case folding, removing stop words can **improve efficiency** but comes at the **cost of losing contextual information**.
- The choice of whether to use stop word removal will depend on the task being performed. For some tasks like **topic modelling** (identifying topics in text), contextual information is not as **important** compared to a task like sentiment analysis where the stop word "not" can change the **sentiment completely**.
- Also note that different libraries have **different** stop word lists so you might want to **tune** your list depending on the application. Spacy's language model has **over 300 stop words**.

Stop word removal

Using spaCy

```
1 import spacy
2 # Load the SpaCy model
3 nlp = spacy.load("en_core_web_sm")
4 # Tokenize
5 s = "The train to London leaves at 10am on Tuesday."
6 doc = nlp(s)
7 # Stop word removal
8 print([t.text for t in doc if not t.is_stop])
```

```
'train', 'London', 'leaves', '10', 'Tuesday', '.']
```

Using NLTK

```
1 import nltk
2 from nltk.corpus import stopwords
3 from nltk.tokenize import word_tokenize
4 # Download the stopwords dataset
5 nltk.download('stopwords')
6 # Sample text
7 text = "This is an example sentence, showing how to remove stop words."
8 # Tokenize the text
9 words = word_tokenize(text)
10 # Get the list of stop words
11 stop_words = set(stopwords.words('english'))
12 # Remove stop words
13 filtered_words = [word for word in words if word not in stop_words]
14 print(filtered_words)
15
```

```
['This', 'example', 'sentence', ',', 'showing', 'remove', 'stop', 'words', '.']
```

Part-of-speech tagging

- Part-of-speech tagging is the method of **classifying how a word is used in a sentence**, for example, **noun, verb, adjective**.
- This is very helpful because it can help us understand the **intent or action** of an ambiguous word. For example, when we say "Hand me a hammer.", the word "hand" is a **verb** (doing word) as opposed to "The hammer is in my hand." where it is a **noun** (thing) and has a different meaning.

"I love pizza."



["pron", "verb", "noun", "punct"]

Part-of-speech tagging Using

```
# Tokenize  
s = "She was the fastest swimmer."  
doc = nlp(s)
```

```
# Part-of-speech  
print([(t.text,t.pos_) for t in doc])
```

```
[('She', 'PRON'), ('was', 'AUX'), ('the', 'DET'), ('fastest', 'ADJ'), ('sw  
immer', 'NOUN'), ('.', 'PUNCT')]
```

A full description of the tags can be found using "spacy.explain".

```
print([(t.pos_, spacy.explain(t.pos_)) for t in doc])
```

```
[('PRON', 'pronoun'), ('AUX', 'auxiliary'), ('DET', 'determiner'), ('ADJ',  
'adjective'), ('NOUN', 'noun'), ('PUNCT', 'punctuation')]
```

Named Entity Recognition

- Named Entity Recognition (**NER**) is the act of tagging **named entities** in text.
- NER can help **categorize and organize** a corpus. It is especially useful, for example, in helping chatbots raise accurate support tickets depending on the customer problem.
- When I say my name is Mohamed and I live in Cairo and work at X company, he explains to me this information and tells me that Mohamed is a person, Cairo is a place, and X is a company.

Olivia Johnson **PERSON**

bought a

Samsung **ORG**

phone for

200 dollars **MONEY**

Named Entity Recognition

```
1 sentence="Apple is looking at buying U.K. startup for $1 billion"
```

```
1 import spacy
2 nlp = spacy.load("en_core_web_sm")
```

```
1 doc1 = nlp(sentence)
2 for ent in doc1.ents:
3     print(ent, " | ", ent.label_, " | ", spacy.explain(ent.label_))
```

Apple | ORG | Companies, agencies, institutions, etc.
U.K. | GPE | Countries, cities, states
\$1 billion | MONEY | Monetary values, including unit

```
1 from spacy import displacy
2
3 displacy.render(doc1, style="ent")
4
```

Apple ORG is looking at buying U.K. GPE startup for \$1 billion MONEY

Spell correction

Spell checker

```
1 #!pip install pyspellchecker
2 from spellchecker import SpellChecker
3 spell = SpellChecker()
4 misspelled = spell.unknown(["cmputr", "watr", "study", "wrte"])
5 for word in misspelled:
6     print(spell.correction(word))
```

write
computer
water

TextBlob library

```
1 ### !pip install textblob
2 from textblob import TextBlob
3
4 a = "cmputr"          # incorrect spelling
5 print("original text: "+str(a))
6
7 b = TextBlob(a)
8
9 # prints the corrected spelling
10 print("corrected text: "+str(b.correct()))
```

original text: cmputr
corrected text: computer